Structural Invariant-Schema Inference and Semantic Verification for Lambda-Free Arrow Graphs

Anonymous

September 2026

Abstract

We present an annotation-free invariant inference mechanism for lambda-free arrow graphs — directed graphs built from four combinators (arr, comp, first, loop) over a signature of base arrows. The graph carries no refinement annotations, proof terms, or user-supplied loop invariants. Our mechanism infers invariant schemas from graph topology and verifies them against base arrow contracts using Z3. The compositional postcondition propagation is proven sound in Lean4: every inferred postcondition holds for every well-typed execution. The invariant domain D is formally defined, and the inference is proven strictly stronger than the trivial sound floor. We also prove that invariants outside D exist and are undetectable (incompleteness). Loop invariant discovery — structural pattern recognition for side-channel, accumulator, and filter loops — is implemented in a Rust prototype and evaluated empirically. Across 72 non-trivial benchmark cells the prototype achieves 100% accuracy: every property the graph should prove is proven (26/26), and every property it should not prove is correctly reported as unknown (46/46), with zero false positives. This contrasts with 37 annotations required by Liquid Types for 19 properties.

1 . I n t r o d u c t i o n P r o g r a m v e r i f i c a t i o n i s t r a d i t i o n a l l y e x p e n s i v e . T o p r o v e t h a t a f u n c t i o n p r e s e r v e s s o r t e d n e s s , m a i n t a i n s a l e n g t h

invariant,
or
satisfies
a
loop
specification,
one
supplies,
in
addition
to
the
program,
a
collection
of
auxiliary
artifacts:
refinement-type
annotations
on
every
sub-expression,
explicit
proof
terms
for
each
verification
condition,
and

-
-
-

most
stubbornly

-

-
-

l o o p
i n v a r i a n t s
t h a t
t h e
p r o g r a m m e r
m u s t
d i s c o v e r
a n d
s t a t e
b y
h a n d .
T h e
a n n o t a t i o n s
o f t e n
o u t w e i g h
t h e
p r o g r a m .
T h e
l o o p - i n v a r i a n t
p r o b l e m
i n
p a r t i c u l a r
i s
t h e
c a n o n i c a l
b o t t l e n e c k :
t h e
i n v a r i a n t
i s
t h e
c r e a t i v e
s t e p
t h a t
n e i t h e r

t h e
t y p e
c h e c k e r
n o r
t h e
t h e o r e m
p r o v e r
c a n
r e l i a b l y
t a k e
o n
i t s
o w n .
T h i s
p a p e r
s t a r t s
f r o m
a
d i f f e r e n t
r e p r e s e n t a t i o n
o f
p r o g r a m s .
A n

-
- a r r o w

g r a p h
i s
a
d i r e c t e d ,
p o r t - t y p e d
g r a p h
b u i l t
f r o m
c o m b i n a t o r s
o v e r
a

signature of base arrows. The calculus is

-
- lambda
- free
-
-

: the grammar has no binder, no variable, and no application. The key observation is that an arrow graph is

- structurally

transparent*: its wiring is

explicit, every wire is inspectable, and the combinator structure determines how information flows. This raises the question:

- Can we achieve Agda-level semantic guarantees
  -
  -
  -

sortedness,

permutation, loop invariants

-
-
-

at TypeScript-level

p r o g r a m m i n g

e r g o n o m i c s ,
b y
d e r i v i n g
p r o o f s
f r o m
t h e
s t r u c t u r e
o f
a
l a m b d a - f r e e

a r r o w
g r a p h
r a t h e r
t h a n
f r o m
u s e r - s u p p l i e d
a n n o t a t i o n s
o r
e x p l i c i t

p r o o f
t e r m s ? *
W e
a r g u e
t h a t
i t
c a n ,
f o r
a
u s e f u l
a n d
c h a r a c t e r i z a b l e
c l a s s

of
programs,
demonstrated
with
a
Rust
prototype
performing
annotation-free
inference
and
a
Lean4
mechanization
establishing
soundness.
The
positioning
is
deliberate:
TypeScript
gives
lightweight
types
with
low
programmer
burden;
Agda
gives
very
strong
proofs
but
makes
proof

construction
explicit.
Arrow
graphs
aim
for
the
sweet
spot
between
them,
as
the
next
subsection
makes
concrete.

The
Sweet
Spot
Consider
the
spectrum
of
programmer
burden
versus
verification
power:
‘
JavaScript:
write

algorithm
$
to$
no
types,
no
proofs,
no
guarantees
TypeScript:
write
algorithm
- types
$
to$
types,
no
proofs,
no
semantic
guarantees
Agda:
write
algorithm
- types
- proofs
$
to$
types
- proofs,
strong
guarantees
Arrow

graphs:
write
algorithm
$
to$
types
•
proofs,
strong
guarantees
‘
JavaScript
requires
nothing
beyond
the
algorithm
but
provides
no
guarantees.
TypeScript
adds
type
annotations
and
gets
basic
types
but
no
semantic
proofs.
Agda
requires
the

programmer
to
construct
explicit
proofs
and
achieves
very
strong
guarantees.
Arrow
graphs
aim
for
the
fourth
position:

-
- the
programming
ergonomics
of
JavaScript
(just
write
the
algorithm,
nothing
else)

-
- with
the
verification
power
of
Agda
(real

theorems
about
behavior).
The
user
does
not
write
types.
The
user
does
not
write
proofs.
The
user
does
not
write
generics.
The
user
does
not
write
annotations.
The
user
just
writes
the
program

-
-
-

a n d
t h e
t y p e ,
t h e
p o l y m o r p h i s m ,
a n d
t h e
p r o o f
a l l
f a l l
o u t
o f
t h e
g r a p h
s t r u c t u r e .
T h i s
i s
p o s s i b l e
b e c a u s e
t h e
g r a p h
s t r u c t u r e
c o n t a i n s
a l l
t h e
i n f o r m a t i o n .
T h e
t y p e
i s
i n
t h e
w i r i n g .
T h e
p r o o f
i s
i n

t h e
w i r i n g .
T h e
p o l y m o r p h i s m
i s
i n
t h e
w i r i n g .
T h e r e
i s
n o t h i n g
t o
d e c l a r e
b e c a u s e
t h e
s t r u c t u r e
a l r e a d y
h a s
e v e r y t h i n g .
T h e
d i f f e r e n c e
f r o m
J a v a S c r i p t
i s
n o t
t h a t
w e
s k i p
v e r i f i c a t i o n

-
-
-

i t
i s
t h a t
w e
d o

f u l l
v e r i f i c a t i o n
b y
r e a d i n g
i t
o f f
t h e
s t r u c t u r e
i n s t e a d
o f
a s k i n g
t h e
u s e r
t o
w r i t e
i t
d o w n .
T h e
k e y
q u e s t i o n
i s
w h e t h e r
t h i s
c a n
b e
a c h i e v e d
w i t h o u t
q u i e t l y
m o v i n g
t h e
p r o o f - w r i t i n g
b u r d e n
i n t o
a n n o t a t i o n s
o r
l i b r a r y

specifications.

JavaScript

TypeScript

Agda

Arrow

graphs

(this

paper)

|

-
-
-
-
-
-
-
-
-
-
-
-
-

What

you

write

Algorithm

Algorithm

-

types

Algorithm

-

types

-

proofs

-

- Algorithm

only

Basic

types

None

Declared

(with

generics)

Declared

(with

universe

params)

-
- Inferred

from

wiring

Prove

sortedness

No

No

Explicit

proof

-
- Inferred

from

graph

Prove

permutation

No

No

Explicit

proof

-
- Inferred
-
-

Polymorphism

Dynamic

Declared

< T >

Declared

‘{ A

B

Set}‘

-
- Inferred

($
alpha$,

$
beta$

from

wiring)

Proof

attached

to

program

No

No

Separate

dependent

term

-
- Same

graph

Composition

Excellent

Excellent

Proofs

compose

explicitly

-
- Composition
-

inferred
proofs
Loop
invariants
External/manual
External/manual
Explicit
dependent
reasoning

-
- Potentially

inferred
by
`loop`
Programmer
burden
Lowest
Low
High

-
- Low

(target)
Guarantees
None
Basic
types
Very
strong

-
- Potentially

very
strong

-

automatic
The
"potentially"
qualifiers

a r e
d e l i b e r a t e :
t h e
l o o p
i n v a r i a n t
d i s c o v e r y
i s
i m p l e m e n t e d
a n d
e m p i r i c a l l y
v a l i d a t e d
b u t
n o t
y e t
f o r m a l l y
p r o v e n
i n
L e a n 4
(S e c t i o n
5 . 5) .
T h e
t a b l e
e x p r e s s e s
t h e

- t a r g e t
-

o f
t h i s
w o r k ,
a n d
t h e
p a p e r
h o n e s t l y
c h a r a c t e r i z e s
h o w
f a r

t h e
c u r r e n t
i m p l e m e n t a t i o n
a n d
f o r m a l i z a t i o n
r e a c h
t o w a r d
i t .

-
- C o n t r i b u t i o n s .
-
-

W e
f r a m e
o u r
c o n t r i b u t i o n
a s

-
- g r a p h
- s t r u c t u r a l

p r o o f
i n f e r e n c e :
t h e
a r r o w
g r a p h
i s
s i m u l t a n e o u s l y
t h e
p r o g r a m ,
t h e
t y p e ,
a n d
t h e
p r o o f
o b j e c t ,
a n d
t h e
i n f e r e n c e

procedure
derives
properties
from
the
graph's
combinator
structure.
This
is
not
refinement
inference
(which
annotates
an
AST)
or
invariant
synthesis
(which
analyzes
loop
semantics)

-
-
-

it
is
proof
inference
from
the
structure
of
a
lambda-free

g r a p h .
T h e
b r o a d
v i s i o n
i s
t h a t
t h e
a r r o w
p r o g r a m
i t s e l f
i s
t h e
i n p u t
t o
a
p r o o f - i n f e r e n c e
p r o c e d u r e ,
a n d
t h e
r e s u l t i n g
p r o o f
i s
c o m p o s i t i o n a l
w i t h
t h e
p r o g r a m
g r a p h .
T h e
n a r r o w
c o n t r i b u t i o n
o f
t h i s
p a p e r
i s
w h a t
w e

a c t u a l l y

p r o v e

a n d

i m p l e m e n t :

1 .

-
- A n n o t a t i o n
- f r e e

s t r u c t u r a l

r e f i n e m e n t

p r o p a g a t i o n

w i t h

s e m a n t i c

v e r i f i c a t i o n .

A

s t r u c t u r a l

m e c h a n i s m

i n f e r s

i n v a r i a n t

s c h e m a s

f r o m

g r a p h

t o p o l o g y

a n d

v e r i f i e s

t h e m

a g a i n s t

b a s e

a r r o w

c o n t r a c t s .

T h e

i n f e r e n c e

i s

s o u n d

(f o r m a l l y

p r o v e n

i n

L e a n 4)
b u t
n o t
c o m p l e t e
-
-
-
i t
d i s c o v e r s
i n v a r i a n t s
w i t h i n
a
s t r u c t u r a l
d o m a i n
$ D $
a n d
r e p o r t s
” u n k n o w n ”
f o r
i n v a r i a n t s
o u t s i d e
$ D $.
2 .
-
- S t r u c t u r a l
i n v a r i a n t - s c h e m a
i n f e r e n c e
f o r
l o o p s .
F o r
‘ l o o p
G ‘
w h e r e
$ G
:
A

t i m e s
C

r i g h t s q u i g a r r o w
B

t i m e s
C $,
t h e
i n f e r e n c e
r e c o g n i z e s
t h e
s t r u c t u r a l
p a t t e r n
o f
$ G $
(s i d e - c h a n n e l ,
a c c u m u l a t o r ,
f i l t e r)
t o
i n f e r
a n
i n v a r i a n t
s c h e m a
$ I (c) $
o n
t h e
f e e d b a c k
c h a n n e l ,
t h e n
v e r i f i e s
i t
w i t h
Z 3 .
N o
u s e r - s u p p l i e d

invariant
is
required
for
the
cases
we
study.
3.

-
- A

formalized
invariant
domain
with
soundness
and
incompleteness.
The
domain
D
is
defined
as
an
inductive
type
in
Lean4.
The
structural
inference
algorithm
is
formalized
and

proven
sound
(inferred_postcondition_sound_full),
strictly
stronger
than
the
trivial
floor
(floor_does_not_imply_inferred),
and
the
existence
of
undetectable
invariants
outside
D
is
proven
(analyze_incompleteness).
4.

-
- An

empirical
characterization
with
100%
accuracy
on
in-domain
properties.
Across
72
non-trivial
benchmark

c e l l s ,
t h e
p r o t o t y p e
p r o v e s
e v e r y
p r o p e r t y
t h e
g r a p h
s h o u l d
p r o v e
(2 6 / 2 6)
a n d
c o r r e c t l y
r e p o r t s
u n k n o w n
f o r
e v e r y
p r o p e r t y
i t
s h o u l d
n o t
(4 6 / 4 6) ,
w i t h
z e r o
f a l s e
p o s i t i v e s .
W e
d o c u m e n t
5
f a i l u r e
c a s e s
r e p r e s e n t i n g
o u t - o f - d o m a i n
i n v a r i a n t s .

-
- F o u n d a t i o n .
-

- This paper builds on a lambda-free arrow-graph calculus whose proof-theoretic foundations
- a Curry-Howard correspondence between graph typing and logical proof, subject reduction, and local cut elimination
- are established in parallel

w o r k
[s e l f - c i t e]
a n d
m e c h a n i z e d
i n
L e a n 4 .
W e
u s e
t h e s e
r e s u l t s
a s
t h e
l o g i c a l
f o u n d a t i o n
f o r
t h e
i n f e r e n c e
m e c h a n i s m
b u t
t h e y
a r e
n o t
c o n t r i b u t i o n s
o f
t h i s
p a p e r .
A p p e n d i x
A
s u m m a r i z e s
t h e
k e y
f a c t s .

-
- A

n o t e
o n

"annotation-free."
By
"annotation-free"
we
mean
that
the
program
-
-
-
the
arrow
graph
-
-
-
carries
no
refinement
annotations.
The
base
arrow
signatures
(sort,
map,
filter,
etc.)
do
carry
known
guarantees,
but
these
are
the

type signatures of primitive operations, not user-supplied annotations on the program. This is analogous to how Hindley-Milner type inference is "annotation-free" despite base operations having typed signatures.

-
- Two layers of inference, both automatic. The refinement

inference
operates
on
top
of
standard
type
inference.
The
user
writes
the
arrow
graph;
the
system
derives
both
layers
automatically:
1.
   •
   • Type
inference
(Hindley-Milner-style):
the
most
general
type
is
derived
from
the
graph
structure.
The

u s e r
n e v e r
w r i t e s
t y p e
s i g n a t u r e s ,
g e n e r i c
p a r a m e t e r s ,
o r
t r a i t
b o u n d s .
m a p
i s
a u t o m a t i c a l l y
p o l y m o r p h i c :
t h e
i n f e r e n c e
e n g i n e
i n t r o d u c e s
t y p e
v a r i a b l e s
$
a l p h a ,

b e t a $
d u r i n g
u n i f i c a t i o n ,
d e r i v i n g
‘ m a p
:
(L i s t
$
a l p h a $
$
t i m e s $
($
a l p h a $
$
l e a d s t o $

$
beta$))
$
leadsto$
List
$
beta$‘
from
the
wiring
alone.
sort
is
automatically
monomorphic:
‘sort
:
List
$
alpha$
$
leadsto$
List
$
alpha$‘
because
the
graph
structure
constrains
input
and
output
to
the
same
type.
The

u s e r
d i d
n o t
s p e c i f y
e i t h e r

-
-
-

t h e
m o s t
g e n e r a l
t y p e
i s
a l w a y s
d e r i v a b l e
f r o m
t h e
w i r i n g .
2 .

-
- R e f i n e m e n t

i n f e r e n c e
(t h i s
p a p e r) :
t h e
m o s t
p r e c i s e
s e m a n t i c
g u a r a n t e e
i s
d e r i v e d
f r o m
t h e
g r a p h
s t r u c t u r e
p l u s
t h e

axiomatic
contracts
on
base
arrows.
sort
derives
Sorted(output),
map
derives
‘LengthEq(input,
output)‘,
filter
derives
‘AllSatisfy(output,
p)
$
land$
output|
$
le$
input|‘

-
-
-

all
with
zero
annotations.
The
user
writes
one
thing

-
-
-

t h e
a r r o w
g r a p h
•
•
•
a n d
g e t s
b o t h
t h e
t y p e
a n d
t h e
p r o o f
f o r
f r e e .
T h i s
i s
t h e
i n v e r s i o n :
i n
t r a d i t i o n a l
s y s t e m s ,
t h e
t y p e
i s
d e c l a r e d
a n d
t h e
p r o o f
i s
c o n s t r u c t e d .
H e r e ,
b o t h
a r e
i n f e r r e d
f r o m

the
same
object:
the
program
structure.

-
- No

generics
syntax
needed.
Generics
syntax
(<T>,
[T],
‘forall
a.‘)
exists
because
statement-based
languages
require
types
to
be
declared
before
use.
Arrow
graphs
invert
this:
the
graph
IS
the

t y p e
d e r i v a t i o n .
T h e
i n f e r e n c e
e n g i n e
r e a d s
t h e
g r a p h ,
i n t r o d u c e s
t y p e
v a r i a b l e s
w h e r e
n e e d e d ,
a n d
d e r i v e s
t h e
m o s t
g e n e r a l
t y p e .
T h e r e
i s
n o t h i n g
t o
d e c l a r e
b e c a u s e
t h e
s t r u c t u r e
a l r e a d y
c o n t a i n s
a l l
t h e
i n f o r m a t i o n .
P o l y m o r p h i s m
i s
n o t
a

feature
you
opt
into

-
-
-

it
is
the
default,
falling
out
of
the
wiring.

2.
Background

-
- The

arrow
calculus.
Hughes
[26]
introduced
arrows
as
a
generalization
of
monads.
Lindley,
Wadler,
and
Yallop

[31]
developed
the
arrow
calculus
as
a
metalanguage
with
four
constructs
and
five
laws.
Our
graph
primitives
correspond
to
these
constructs;
the
key
difference
is
that
we
take
them
as
a

-
- graph

grammar
rather
than

a
lambda-calculus
extension.
Categorically,
arrows
are
monoids
in
profunctor
categories
[24,
25]:
`comp`
is
monoid
multiplication,
`first`
is
tensorial
strength,
and
`loop`
is
the
trace
of
a
traced
monoidal
category
[27].

-
- Refinement
types
and
Liquid

Types.
Freeman
and
Pfenning
[17]
introduced
refinement
types
for
ML.
Liquid
Types
[38]
extend
Hindley-Milner
inference
with
refinement
types
using
SMT
solving,
but
still
require
annotations
on
recursive
functions
and
loops.
Our
work
differs
in
that

graph structure (not an AST) is the substrate for inference, and loop invariant schemas are inferred by structural analysis on the graph.

-
- Abstract interpretation. Cousot and Cousot [10, 11] developed the dominant framework for loop

invariant
inference.
Our
structural
inference
is
inspired
by
abstract
interpretation
but
operates
on
graph
structure
rather
than
program
syntax.
We
do
not
claim
a
formal
duality;
the
structural
analysis
provides
candidate
schemas,
and
the
SMT
checker

p r o v i d e s
s o u n d n e s s
v i a
s e m a n t i c
v e r i f i c a t i o n .

3 .
T h e
A r r o w - G r a p h
C a l c u l u s

3 . 1
P r i m i t i v e s
a n d
S e m a n t i c s

A n
a r r o w
g r a p h
i s
a
f i n i t e ,
d i r e c t e d ,
p o r t - t y p e d
g r a p h
b u i l t
f r o m
c o m b i n a t o r s
o v e r
a
s i g n a t u r e
$
S i g m a $
o f

```
b a s e
a r r o w s
$
v a r p h i
:
A

t o
B $ :
$ $

b e g i n { a l i g n e d }

t e x t { T y p e s : }

q u a d
&
A ,
B ,
C

i n

t e x t { T y }
: =

a l p h a

m i d
A

t i m e s
B

m i d
A
   •
B

m i d
```

```
t o p

m i d

b o t



t e x t { G r a p h s : }

q u a d
&
G ,
H

i n

t e x t { G r a p h }
: =

t e x t s f { i d } _ A

m i d

t e x t s f { c o m p } ( G ,
H )

m i d

t e x t s f { f i r s t } ( G )

m i d

t e x t s f { l o o p } ( G )

m i d

t e x t s f { a r r } (
v a r p h i )

e n d { a l i g n e d }
```

$$
The
calculus
also
includes
left(G)
for
sum
types
($A
•
C
rightsquigarrow
B
•
C$),
but
the
inference
mechanism
does
not
use
it
•
•
•
left
is
always
treated
conservatively
(Section
4.3).
We
include

it
for
completeness
of
the
calculus
but
focus
on
the
four
combinators
that
drive
the
inference:
`arr`,
`comp`,
`first`,
and
`loop`.

-
- No

lambda-abstraction,
no
application,
no
variable
appears.
The
graph
reading
is
recovered
by
viewing

comp
as
edge-sequencing,
first
as
functorial-boxing
on
products,
and
loop
as
a
back-edge
(cycle).
The
operational
semantics
is
standard.
Execution
$G

cdot
a

Longrightarrow
v$
is
defined
by:
identity
returns
its
input;
composition
runs
the

left
then
the
right;
first
runs
the
sub-graph
on
the
first
component
and
passes
the
second
through;
arr$(
varphi)$
applies
the
base
arrow;
and
loop
iterates
the
step
graph
on
the
feedback
channel
until
it
stabilizes.

3.2
Foundation:
Graphs
as
Proof
Objects
In
parallel
work
[self-cite],
we
show
that
arrow
graphs
satisfy
a
Curry-Howard
correspondence:
the
typing
judgment
‘TyOf
G‘
and
the
proof
judgment
‘PrOf
G‘
are
bijections,
subject

reduction holds, and cut elimination is a local graph rewrite. These results are mechanized in Lean4 (`ProofGraphs.lean`, `ProofAdvantage.lean`) and serve as the foundation for the inference mechanism. The key fact used in this paper:

-

- Subject

Reduction
(Lean4).
If
$
vdash
G
A

rightsquigarrow
B$
and

$G

cdot
a

Longrightarrow
v$
with
$a
:
A$,
then
$v
:
B$.
This
is
used
in
the
soundness
proof
of
the

inference algorithm (Section 4.4) to establish that intermediate values in compositions are well-typed. Full details are in Appendix A.

The Curry-Howard correspondence is not just a theoretical curiosity

-
-
-

it is the

- enabling

condition*
for
the
inference
mechanism.
Because
the
graph
IS
a
proof
object
(its
typing
derivation
is
a
proof
derivation),
the
inference
procedure
can
extract
proof-relevant
information
from
the
graph
structure
directly.
When
we
infer
Sorted
from

‘comp(arr(filter(p)),
arr(sort))‘,
we
are
reading
the
proof
structure
off
the
graph:
the
cut
rule
(composition)
combines
the
axiom
(arr)
for
filter
with
the
axiom
for
sort,
and
the
conjunction
of
their
guarantees
is
the
inferred
postcondition.

This
is
what
makes
the
inference

- proof

inference*
rather
than

- type

inference*:
we
are
not
just
assigning
types,
we
are
reconstructing
a
proof
from
the
proof
object's
structure.

4.
Refinement
Propagation

4.1

Refinement

Types

A

-
- refinement

type

is

written

$
{x

:

T

mid

P(x)
}$,

denoting

the

subset

of

values

of

type

T

satisfying

predicate

P.

A

refinement

arrow

$
{x

:

A

m i d

P (x)
}

r i g h t s q u i g a r r o w

{ y

:

B

m i d

Q (x ,

y)
} $

d e n o t e s

a

c o m p u t a t i o n

t h a t ,

g i v e n

a n

i n p u t

s a t i s f y i n g

$ P $,

p r o d u c e s

a n

o u t p u t

s a t i s f y i n g

$ Q $.

4 . 2

T h e

K e y

I d e a :

R e f i n e m e n t s

Flow
from
Structure
The
central
claim
is
that

-
- refinements

flow
from
base-arrow
signatures
plus
graph
structure,
without
user
annotations
on
the
graph.
The
signature
$
Sigma$
is
the
language
definition:
each
base
arrow
$
varphi$
carries

a
known
pre-/post-condition,
like
the
type
of
+
or
`cons`.
The
graph
is
the
program

- •
- •
- •

it
carries
no
type
information,
no
refinement
annotation,
no
proof
term.
The
checker
traverses
the
graph,
propagates
refinements
through

each
combinator,
and
infers
the
output
refinement
from
the
input
refinement
and
the
graph
structure.

4.2.1
The
Inversion:
The
Program
Is
the
Proof
The
mechanism
is
not
heuristic,
statistical,
or
search-based.

It
is
a
- deterministic
structural
derivation :
1.
- Read
the
graph
-
-
-
which
combinators
are
wired
together?
2.
- Look
up
the
axioms
-
-
-
what
do
the
leaf
arrows
guarantee?
3.
- Compose
-

-
-
-
-

comp
conjoins
guarantees,
first
preserves
the
side
channel,
choice
branches
through
both
arms,
loop
finds
a
fixpoint.
4.

-
- Check

implications

-
-
-

can
we
prove
$
textsf{guarantee}

implies

textsf{postcondition}$?
There
is

n o
m a c h i n e
l e a r n i n g ,
n o
s e a r c h ,
n o
t r i a l - a n d - e r r o r .
T h e
p r o o f
i s

-
- s t r u c t u r a l l y

d e t e r m i n e d
b y
t h e
w i r i n g .
C h a n g e
t h e
w i r i n g
a n d
y o u
g e t
a
d i f f e r e n t
p r o o f .
T h e
g r a p h

- i s
-

t h e
p r o o f .
T h i s
i s
t h e
i n v e r s i o n
t h a t
d i s t i n g u i s h e s

proof
inference
from
proof
construction.
In
Agda,
you
write
the
proof.
In
Coq,
you
write
the
tactic
script.
In
test-based
verification,
you
write
the
assertions.
Here,
you
write

-
- the

program ,
and
the
proof
falls
out

f o r
f r e e .
T h e
p r o g r a m
i s
t h e
p r o o f ,
a n d
t h e
c h e c k e r
r e a d s
t h e
p r o o f
o f f
t h e
s t r u c t u r e .
T h i s
i s
w h a t
m a k e s
t h e
a p p r o a c h
- p r o o f
i n f e r e n c e *
r a t h e r
t h a n
- t y p e
i n f e r e n c e * :
w e
a r e
n o t
j u s t
a s s i g n i n g
t y p e s ,
w e
a r e

reconstructing
a
proof
from
the
proof
object's
structure.
The
Curry-Howard
correspondence
(Section
3.2)
is
the
enabling
condition

-
-
-

because
the
graph
IS
a
proof
object,
its
typing
derivation
is
a
proof
derivation,
and
the
inference

procedure
extracts
proof-relevant
information
from
the
graph
structure
directly.

4.2.2
The
Trusted
Foundation
The
only
stated
contracts
are
the

-
- axioms
-
-

:
trivial
structural
facts
about
irreducible
primitive
operations.
cons

adds
1
to
length.
`compare`
returns
a
value
$
leq
1$.
`alloc`
produces
an
empty
list.
These
are
not
deep
properties

-
-
-

they
are
the
definitions
of
the
operations.
Given
this
small
trusted
foundation,
arbitrary

composed
arrow
graphs
acquire
derived
semantic
guarantees
automatically.
The
ratio
is
the
value
proposition:
$
sim
10$
axiomatic
contracts
enable
infinitely
many
programs
to
be
verified
with
zero
annotations.

4.3
The
Inferred
Postcondition

The
function
`inferred_postcondition$(G)$`
computes
a
predicate
on
the
output
value
of
G
by
structural
recursion
on
the
graph:

- `id` $ to$ `True` (no constraint beyond input = output).
- `arr$(`varphi`)$` $ to$ `sig_guarantee$(`varphi`)$` (the

base
arrow's
output
contract).
  •
comp$(G,
H)$
$
to$
inferred_postcondition(H)
(propagate
through
H;
H
produces
the
final
output).
  •
first(G)
$
to$
True
(conservative:
side
channel
unconstrained).
  •
loop(G)
$
to$
True
(conservative:
floor
for
feedback;
see

Section
5).

- `left$(G)$`
$
to$
`True`
(conservative:
not
used
by
the
inference).
This
mirrors
the
Rust
`infer_r`
function.
For
`comp$(G,`
H)$,
the
postcondition
of
the
composition
is
the
postcondition
of
H:
if
H
guarantees
Q

o f
i t s
o u t p u t ,
t h e n
c o m p $ (G ,
H) $
g u a r a n t e e s
$ Q $
o f
i t s
o u t p u t .

4 . 4
S o u n d n e s s

-
- T h e o r e m
(S t r u c t u r a l
I n f e r e n c e
S o u n d n e s s ,
L e a n 4
-
-
-
i n f e r r e d _ p o s t c o n d i t i o n _ s o u n d _ f u l l) .
- L e t
$
S i g m a $
b e
a
s i g n a t u r e
w i t h
s o u n d
d e n o t a t i o n
a n d
s o u n d

guarantees:
for
every
$
varphi
:
A

to
B$
and
$a
:
A$,
sig_guarantee$(
varphi)$
holds
of
$
textsf{den}(
varphi,
a)$.
For
every
well-typed
graph
G
with
$
textsf{TyOf}(G)$,
every
$a
:
A$,
and
every
v
with

```
$G

cdot
a

Longrightarrow
v$,
the
inferred
postcondition
holds
of
$v$:*
$$

textsf{inferred
_postcondition}(G)(v).
$$
```

- Proof

```
is
by
induction
on
the
evaluation
relation
Eval.
The
eComp
case
uses
subject
reduction
on
$G$
to
obtain
```

the
intermediate
value's
type,
feeding
the
inductive
hypothesis
for
H.
The
`eArr`
case
uses
the
signature
soundness
hypothesis.
The
conservative
cases
(`eFirst`,
`eLeft`,
`eLoop`)
hold
trivially
since
the
postcondition
is
`True`.
Zero
`sorry`*
(`StructuralInference.lean`).

4.5
The
Invariant
Domain
D
The
invariant
domain
D
is
the
smallest
set
of
predicates
(on
values)
containing:
(a)
base
arrow
guarantees
(sig_guarantee),
(b)
True,
(c)
conjunctions
of
predicates
in
D
($P

wedge

Q $
whenever
$ P ,
Q

in
D $) ,
and
(d)
predicates
lifted
through
`f i r s t`
(component-wise ,
`o n F i r s t`) .
$ D $
does

-
- not
-
-

contain
arbitrary
arithmetic
predicates ,
predicates
requiring
induction
on
iteration
count ,
or
predicates
not
expressible
in
the

signature
language.
It
is
formalized
in
Lean4
as
the
inductive
type
InvariantDomain
(InvariantDomain.lean).

-
- Theorem
(Domain
Membership,
Lean4
-
-
-
inferred_postcondition_in_domain).
- For
every
graph
G,
the
inferred
postcondition
is
a
member
of
the
invariant
domain
D.

Zero

sorry.*

-
- Theorem

(Strictly

Stronger

than

the

Floor,

Lean4

-
-
-

floor_does_not_imply_inferred).

- For

any

base

arrow

$
varphi$

whose

guarantee

is

non-trivial

(there

exists

v

with

$
neg
,
textsf{sig
_guarantee}(
varphi)(v)$),

the

sound

floor

True

does

not
imply
the
inferred
postcondition.
That
is,
the
structural
inference
is
strictly
stronger
than
the
trivial
sound
floor.
Zero
sorry.*

-
- Theorem
(Incompleteness

-
-
-
Invariants
Outside
D,
Lean4

-
-
-
analyze_incompleteness).

- There
exists
a
graph

G
and
a
predicate
I
such
that
(1)
G
preserves
I
(if
$I(c)$
holds
before
a
step,
$I(c')$
holds
after),
but
(2)
I

notin
D$

- •
- •
- •

no
derivation
in
the
invariant
domain
produces
I.

Concretely:
with
an
empty
signature,
the
identity
loop
body
preserves
the
invariant
$I
=

textsf{False}$
(vacuously),
but
$
textsf{False}$
cannot
be
derived
in
D
from
an
empty
signature.
This
proves
the
structural
approach
is
inherently
incomplete:

true
invariants
exist
outside
D.
Zero
sorry.*

4.6
Concrete
Examples
Consider
a
signature
with
sort
guaranteeing
Sorted
$
wedge$
IsPerm,
map(f)
guaranteeing
IsList
$
wedge$
LengthPreserved,
and
filter(p)
guaranteeing
IsList
$
wedge$
AllSatisfy(p):

Graph
Inferred
output
refinement

|

-
-
-
-

arr(sort)
Sorted
$
wedge$
IsPerm
arr(map(f))
IsList
$
wedge$
LengthPreserved
‘comp(arr(filter(p)),
arr(sort))‘
AllSatisfy(p)
$
wedge$
Sorted
first(arr(sort))
(Sorted
$
wedge$
IsPerm,
Q)
The
third
row
illustrates
compositional
inference:
filter
produces
AllSatisfy(p),
which
flows

into
`sort`'s
precondition;
the
conjunction
is
inferred
without
annotation

-
-
-

the
composition
of
two
base-arrow
postconditions
via
the
cut
rule.

4.7 Worked Example: Insertion Sort

We walk through the full inference

pipeline
for
insertion
sort,
expressed
as
an
arrow
graph.
The
graph
is:
‘
loop(comp(comp(first(head *and* tail),
first(insert)),
consume_one))
‘

-
- Step
1
Structural
analysis.
The
inference
traverses
the
graph:

-
`head_and_tail`
splits
the
input
list
into
head

-

tail.

- `insert` inserts the head into the accumulator C, maintaining sorted order. Its signature guarantees `Sorted$(C')$`.
- `consume_one` removes one element from the input, decrementing its length. Its signature guarantees $|C'|$ `leq` $C|$.
-

The
`first`
combinators
thread
the
accumulator
C
through
unchanged
while
the
input
is
consumed.

-
- Step

2

Candidate
invariant
generation.
The
`analyze`
function
recognizes
the
accumulator
pattern:
`insert`'s
guarantee
`Sorted`
is
threaded
through
the
loop
body.

The
candidate
invariant
is
$I(C)
=

textsf{Sorted}(C)$.

-
- Step

3

Fixpoint
iteration.

-
$I_0
=

textsf{True}$
(initial
floor).

-
$I_1
=

textsf{Sorted}(C)$
(from
`insert`'s
guarantee).

-
$I_2
=

textsf{Sorted}(C)

wedge

textsf{Sorted}(C)$
(fixpoint

reached

-
-
-

$I_2

equiv
I_1$).

-
- Step
4
Z3
verification.
The
candidate
Sorted(C)
is
verified
against
the
base
arrow
contracts:
if
C
is
sorted
and
insert
maintains
sortedness,
then
C'
is
sorted.
Z3
confirms:

-
- verified
-
- .
-
- Step

5

Output refinement. The output refinement is $\mathtt{Sorted}(\textsf{output})$, inferred from the graph structure alone

-
-
-

zero user annotations.

-
- Step

6

Termination. The termination analysis discovers that `consume_one`

decrements
$|C|$
by
1
each
iteration,
bounded
below
by
0.
A
well-founded
ranking
function
is
synthesized:

-
- termination

guaranteed .

5. Structural Invariant-Schema Inference for Loops

5.1 The Problem

The hard case

i s
l o o p .
A
l o o p
‘ l o o p
G ‘
w h e r e
$ G
:
A

t i m e s
C

r i g h t s q u i g a r r o w
B

t i m e s
C $
f e e d s
t h e
$ C $ - o u t p u t
b a c k
a s
t h e
$ C $ - i n p u t
a n d
i t e r a t e s
u n t i l
t h e
f e e d b a c k
c h a n n e l
s t a b i l i z e s .
T o
i n f e r
a
r e f i n e m e n t

o n
t h e
o u t p u t ,
w e
n e e d
a n
- i n v a r i a n t
$ I (c) $
o n
t h e
f e e d b a c k
c h a n n e l
a
p r e d i c a t e
s u c h
t h a t
$ G $
p r e s e r v e s
$ I $:
i f
$ I (c) $
h o l d s
b e f o r e
t h e
s t e p ,
t h e n
$ I (c ') $
h o l d s
a f t e r ,
w h e r e
$ (b ,
c ')

=
G(a,
c)$.
This
is
the
classical
loop-invariant
inference
problem,
restated
on
graph
structure:
the
loop
body
is
the
graph
G,
the
feedback
channel
C
is
the
mutable
state,
and
I
is
a
predicate
on
C

that
each
step
preserves.

5.2 Structural Pattern Recognition

The
`analyze`
phase
inspects
the

- structure
-

of
G
to
determine
its
effect
on
the
feedback
channel
C.
The
graph
wiring
makes
this
a
structural

case analysis, not semantic reasoning over an opaque expression:

-
-
- Side
- channel

:

```
loop(first(H)).
```

The body acts on the first component via H and passes C through unchanged. C is never inspected or modified. The candidate

is
`True`.
The
output
refinement
is
`inferred_postcondition$(H)$`
on
the
first
component.

-
-
- Accumulator

:
`loop(comp(map_step,
first(id)))`.
The
`map_step`
processes
an
element
and
updates
C;
`first(id)`
keeps
the
previous
C
available.
If
`map_step`'s
signature
encodes
that
it

adds
to
C
(e.g.,
postcondition
$c'
=
c
•
a$),
the
invariant
is
$I(c)
=
(c
=

sum

text{processed})$.
•
•
• Filter
:
‘loop(comp(filter_step,
id))‘.
The
filter_step
removes
elements
failing
a
predicate
p.
The
invariant

is
`AllSatisfy$(C,`
p)$.

-
-
- Sortedness

:
‘loop(comp(insert_step,
first(id)))‘.
The
`insert_step`
inserts
one
element
maintaining
sorted
order.
The
invariant
is
`Sorted$(C)$`.
This
is
insertion
sort
expressed
as
a
loop
(Section
4.7).

5.3
The

Invariant
Lattice
and
Transfer
Function
We
formalize
the
invariant-schema
inference
as
iteration
over
a
lattice
$(D,

le)$
where
D
is
the
invariant
domain
(Section
4.5),
$I

le
J$
means
$J

Rightarrow
I$
(stronger
predicates

are
higher),
bottom
is
`True`
(the
trivial
invariant),
and
join
is
conjunction
(strengthening).
The
transfer
function
`transfer$(G,`
`I)$`
computes
the
weakest
pre-invariant.
The
least
fixpoint
`lfp(transfer)`
is
the
strongest
invariant
discoverable
by
the
iteration.
This
is

mechanized
in
Lean4
(InvariantDomain.lean):
the
lattice
ordering
DLe,
bottom
DBottom,
join
DJoin,
the
transfer
function
transfer,
the
fixpoint
predicate
IsFixpoint,
and
the
theorems:

-
-
- True_is_fixpoint
-
-

(Lean4):
the
sound
floor
True
is
always
a
fixpoint

of
the
transfer.
Zero
sorry.

-
-
- lfp_is_invariant
-
-

(Lean4):
every
fixpoint
of
the
transfer
is
a
preserved
invariant

-
-
-

if
I
is
a
fixpoint
and
$G

cdot
(a,
c)

Longrightarrow
(v,
c')$
with

$I(c)$,
then
$I(c')$.
Zero
sorry.

5.4
What
the
Rust
Implementation
Does
The
Rust
prototype
implements
the
full
pipeline:
structural
candidate
generation
from
graph
patterns,
followed
by
Z3-backed
SMT
verification
of
each
candidate

against
base
arrow
contracts.
The
`discover_invariant`
function
performs
structural
case
analysis
on
the
loop
body
and
produces
a
candidate
invariant
schema,
which
is
then
semantically
verified.
The
checker
is

-
- conservative
-
-

:
it
may
return

`Unknown`
for
true
implications
it
cannot
discharge,
but
it
never
returns
`Verified`
for
a
false
one.
This
asymmetry
makes
the
inference
sound

-
-
-

every
property
reported
as
verified
is
genuinely
true.

5.5
What
Lean4
Proves
vs.
the
Gap
The
Lean4
formalization
proves
the
soundness
of

-
- compositional

postcondition
propagation
(`inferred_postcondition_sound_full`).
For
loops,
the
formalized
`inferred_postcondition`
returns
`True`
(the
conservative
floor),
and
the
lattice/transfer
machinery
is

defined
with
`True`
proven
as
a
fixpoint.
The
-
- loop
invariant
discovery
-
-
-
structural
pattern
recognition
that
strengthens
beyond
`True`
for
side-channel,
accumulator,
and
filter
loops
-
-
-
is
implemented
in
Rust
but
-
- not

formally proven in Lean4. Formalizing the strengthened algorithm and proving its algorithmic completeness is future work.

5.6 Termination Analysis The Rust prototype includes a termination analysis binary (8 tests) that discovers decreasing

measures
for
loops
-
-
-

a
well-founded
ranking
function
on
the
feedback
channel.
This
is
separate
from
the
invariant:
the
inference
proves
- conditional
-

invariants
(if
the
loop
reaches
a
fixpoint,
the
invariant
holds),
while
termination

analysis proves the loop does reach a fixpoint. Termination is implemented but not formalized in Lean4.

6. Implementation and Evaluation

6.1 The Rust Prototype

The Rust prototype (22 files, 22,030

lines)
implements
the
full
inference
pipeline
with
Z3-backed
SMT
verification
(subprocess
invocation
with
a
structural
fallback).
It
comprises
six
binaries:
`refinement_inference`
(20
tests),
`real_programs`
(5
programs),
`quantitative_comparison`
(15
benchmarks),
`termination_analysis`
(8
tests),
`liquid_types_comparison`
(10
examples),
and

evaluation
(90-cell
grid).
The
RefTy
type
models
$
{x
:
T

mid
P(x)
}$
with
a
predicate
algebra,
mirroring
the
RTy
inductive
of
RefinementProofGraphs.lean.

6.2
The
Lean4
Formalization
The
Lean4
mechanization
of

the
inference
mechanism
(4
files,
1,098
lines,
zero
sorry)
establishes:
RefinementProofGraphs.lean
(RTy,
RHasType,
preserves_comp,
preserves_first),
RefinementCharacterization.lean
(completeness/incompleteness
statements),
InvariantDomain.lean
(domain
D,
lattice,
transfer,
True_is_fixpoint,
lfp_is_invariant,
floor_not_complete,
analyze_incompleteness),
and
StructuralInference.lean
(inferred_postcondition_sound_full,
inferred_postcondition_in_domain,
floor_does_not_imply_inferred).
An
additional
5,003
lines

of
Lean4
mechanization
(ProofGraphs.lean,
ProofAdvantage.lean,
etc.)
establish
the
foundational
Curry-Howard
correspondence,
subject
reduction,
and
cut
elimination
used
as
the
logical
basis
for
this
paper.
These
are
described
in
parallel
work
[self-cite]
and
summarized
in
Appendix
A.

6.3 Lean4 vs. Rust: What Is Formal vs. What Is Empirical

Component | Lean4 (formal) | Rust (empirical)

- | -
- | -
- | -
- | -
- | -
- | -
- | -
- | -

Subject reduction (foundation) | Proven [self-cite] | N/A

Refinement typing

(RTy,
RHasType)
Defined
  •
sound
N/A
preserves_comp
/
preserves_first
Proven
N/A
Invariant
domain
D
Defined
N/A
Structural
inference
soundness
Proven
(inferred_postcondition_sound_full)
N/A
Strictly
stronger
than
floor
Proven
(floor_does_not_imply_inferred)
N/A
Incompleteness
(invariants
outside
D)
Proven
(analyze_incompleteness)
N/A

L a t t i c e

•

t r a n s f e r

f u n c t i o n

D e f i n e d

N / A

A c t u a l

i n f e r e n c e

a l g o r i t h m

D e f i n e d

(f l o o r)

I m p l e m e n t e d

(s t r u c t u r a l

•

Z 3)

Z 3

v e r i f i c a t i o n

N / A

I m p l e m e n t e d

L o o p

i n v a r i a n t

d i s c o v e r y

N / A

I m p l e m e n t e d

(s t r u c t u r a l

p a t t e r n s)

T e r m i n a t i o n

a n a l y s i s

N / A

I m p l e m e n t e d

R e a l

p r o g r a m s

N / A

5

p r o g r a m s

C o v e r a g e

/
s c a l i n g
/
f a i l u r e s
N / A
9 0 - c e l l
g r i d
T h e
L e a n 4
f o r m a l i z a t i o n
p r o v e s
t h e
s o u n d n e s s
o f
t h e
i n f e r e n c e
f r a m e w o r k ;
t h e
R u s t
i m p l e m e n t a t i o n
d e m o n s t r a t e s
t h e
a l g o r i t h m
i n
p r a c t i c e .
F o r m a l
t h e o r e m s
e s t a b l i s h
w h a t

- c a n
-

b e
p r o v e n ;
t h e
i m p l e m e n t a t i o n
s h o w s

w h a t

- i s
-

p r o v e n .

6 . 4 E m p i r i c a l R e s u l t s

-
- A n n o t a t i o n

b u r d e n .

A r r o w g r a p h s r e q u i r e

-
- 0

u s e r a n n o t a t i o n s t o p r o v e 1 9 p r o p e r t i e s , w h i l e L i q u i d T y p e s r e q u i r e

-
- 3 7

a n n o t a t i o n s (3 2 r e f i n e m e n t

-

5 l o o p i n v a r i a n t s)

for
the
same
19
properties.
Base
arrow
signatures
carry
guarantees,
but
these
are
the
language
definition,
not
per-program
annotations.

-
- Coverage

accuracy.
The
evaluation
grid
comprises
18
graph
structures
tested
against
5
properties
each
(90
cells

total,
72
non-trivial).
For
each
non-trivial
cell,
we
classify
whether
the
graph
- should
-
prove
the
property
(based
on
its
structure
and
base
arrow
signatures)
or
- should
not*:
Category
Count
Result

|

-
-
-
-

-
-
-
-

Should
prove
(true
positives)
26
26
proven
(100%)
Should
not
prove
(true
negatives)
46
46
unknown
(100%)
False
positives
0

-
-
-

False
negatives
(misses)
0

-
-
-
-
- Overall
accuracy

-

- 7 2
-
-
-
- 1 0 0 %
-
-

T h e
i n f e r e n c e
p r o v e s
e v e r y
p r o p e r t y
i t
s h o u l d
a n d
c o r r e c t l y
r e p o r t s
" u n k n o w n "
f o r
e v e r y
p r o p e r t y
i t
s h o u l d
n o t .
Z e r o
f a l s e
p o s i t i v e s
c o n f i r m s
s o u n d n e s s
e m p i r i c a l l y ;
z e r o
m i s s e s
c o n f i r m s
c o m p l e t e n e s s
r e l a t i v e
t o
t h e
d o m a i n

D.

-
- Scaling.
-
-

Inference time scales linearly in graph size and loop nesting depth, as measured by 15 benchmarks varying graph size from 3 to 50 nodes and loop depth from 0 to 3.

-

- Real programs. All five real programs (insertion sort, `map`, `filter`, `fold/sum`, pipeline) are built from arrow primitives and verified annotation-free. Insertion sort is `loop(comp(comp(first(head *and* tail), first(insert)), consume_one))` with the `Sorted` invariant inferred from `insert`'s signature (Section 4.7).

6.5
Failure
Cases
(Out-of-Domain
Invariants)
We
document
five
failure
cases
representing
invariants
outside
D

-
-
-

properties
the
inference
cannot
discover,
consistent
with
the
incompleteness
theorem
(analyze_incompleteness):
1.

-
- Arithmetic

accumulators.
A
loop

c o m p u t i n g
$ c ’
=
2 c
•
1 $
h a s
t h e
i n v a r i a n t
$ c
=
2 ^ n
•
1 $,
b u t
u n l e s s
a
b a s e
a r r o w
e n c o d e s
t h i s
a r i t h m e t i c ,
t h e
s t r u c t u r a l
a n a l y s i s
h a s
n o
c a n d i d a t e
t o
p r o p o s e .
2 .
•
• P e r m u t a t i o n .
•
•

”Output
is
a
permutation
of
input”
requires
relational
reasoning
not
visible
in
graph
wiring
alone.
3.

-
- Non
- terminating

loops.
The
inference
proves
conditional
invariants;
it
does
not
prove
termination
for
loops
without
a
decreasing
measure.
4.

-
- Cross
- channel

invariants.

Properties

relating

multiple

feedback

channels

(e.g.,

"channels

C_1

and

C_2

have

equal

length")

require

relational

reasoning

across

channels.

5.

-
- Iteration
- count

reasoning.

Invariants

depending

on

- how

many

times*

the

loop

has

executed

a r e
b e y o n d
s t r u c t u r a l
a n a l y s i s
-
-
-
t h e
g r a p h
s h o w s
- h o w
-
$ C $
i s
u p d a t e d ,
n o t
- h o w
m a n y
t i m e s * .
T h i s
i s
-
- i n c o m p l e t e n e s s ,
n o t
u n s o u n d n e s s .
T h e
i n f e r e n c e
n e v e r
r e p o r t s
a
f a l s e
p r o p e r t y ;
i t
f a i l s
t o
r e p o r t
s o m e

true
ones.
The
`analyze_incompleteness`
theorem
(Section
4.5)
formally
proves
that
such
invariants
exist
outside
D.

7.
Related
Work

-
- Liquid

Types.
Rondon,
Kawaguchi,
and
Jhala
[38]
extend
Hindley-Milner
inference
with
refinement
types
using
SMT

solving, but still require annotations on recursive functions and loops. Vazou et al. [41] demonstrate practical refinement typing in Liquid Haskell, but still require annotations on recursive functions. Our work targets the same class of properties

but uses graph structure (not an AST) as the inference substrate, and loop invariant schemas are inferred by structural analysis on the graph. The

0

vs-37 annotation comparison (Section 6.4) quantifies this difference.

-
- Abstract

interpretation.
Cousot
and
Cousot
[10,
11]
developed
the
dominant
framework
for
loop
invariant
inference,
analyzing
the

- semantics
-

of
the
loop
body.
Graf
and
Saidi
[21]
introduced
predicate
abstraction.
Our
approach
infers
invariant

- schemas
-

by
analyzing
the
- structure
-

of
the
loop
body
(a
graph):
the
wiring
of
G
in
‘loop
G‘
explicitly
shows
how
the
feedback
channel
is
used,
determining
the
invariant
schema
class.
This
is
complementary
to
abstract

interpretation, not a replacement.

-
- Data
- driven

loop invariant inference. Sharma et al. [40] use data-driven techniques (machine learning over test executions) to infer loop invariants. Our approach is structural rather than data-driven: the invariant schema is

d e t e r m i n e d
b y
t h e
g r a p h
w i r i n g ,
n o t
b y
o b s e r v i n g
e x e c u t i o n s .

- A r r o w
c a l c u l u s .
H u g h e s
[2 6]
i n t r o d u c e d
a r r o w s ;
L i n d l e y ,
W a d l e r ,
a n d
Y a l l o p
[3 1]
d e v e l o p e d
t h e
a r r o w
c a l c u l u s .
H e u n e n
a n d
J a c o b s
[2 4]
a n d
J a c o b s ,
H e u n e n ,
a n d
H a s u o
[2 5]
e s t a b l i s h e d

the
categorical
semantics.
Recent
work
continues
to
develop
the
semantic
foundations
of
arrows
[39].
Our
four
primitives
are
those
of
the
arrow
calculus
(minus
`left`,
which
is
demoted);
our
contribution
is
taking
them
as
a
graph

g r a m m a r
a n d
d e v e l o p i n g
t h e
i n f e r e n c e
m e c h a n i s m .

-
- H o a r e

l o g i c
f o r
a r r o w s .
A l v e s ,
F e r n a n d e z ,
a n d
M a c k i e
[1]
b u i l d
a
g r a p h i c a l
c a l c u l u s
o f
p r o o f s
f o r
i n t u i t i o n i s t i c
l o g i c
/
l a m b d a - c a l c u l u s .
T h e i r
g r a p h s
a r e
a

- n o t a t i o n
-

f o r
l a m b d a - t e r m s ;
o u r s
a r e

a

- primitive

syntax*

with

no

lambda

underneath.

They

do

not

mechanize

the

correspondence,

prove

subject

reduction,

or

address

type

inference.

-
- Graph
- based

proof

systems.

Acclavio,

Horne,

and

Straßburger

[43]

develop

an

analytic

proof

system

on

graphs,

and
Kurashige
et
al.
[42]
use

e

graphs
for
lemma
discovery
in
equational
proofs.
Both
use
graph
structure
for
proof
manipulation,
but
neither
targets
program
verification
or
invariant
inference.
Our
work
uses
graph
structure
as
the

substrate
for
- inferring
-

program
properties,
not
for
proof
search
or
lemma
discovery.

-
- SMT
- based

verification.
Flanagan
and
Qadeer
[16]
and
Graf
and
Saidi
[21]
use
SMT
solving
for
predicate
abstraction
and
software
verification.
Our
Z3-backed

checker follows the standard sound-but-incomplete paradigm; the difference is that candidates come from structural analysis of the graph, not from predicate-abstraction heuristics or user annotations.

8. Limitations and Future Work

-
- Loop invariant discovery

not
formally
proven.
Lean4
proves
the
soundness
of
compositional
postcondition
propagation
(inferred_postcondition_sound_full).
The
loop-specific
strengthening

-
-
-

structural
pattern
recognition
for
side-channel,
accumulator,
and
filter
loops

-
-
-

is
implemented
in
Rust
but
not
formalized.

The formalized `inferred_postcondition` returns `True` for `loop` (the conservative floor). Formalizing the strengthened algorithm and proving its algorithmic completeness is the primary item of future work.

-
- Algorithmic completeness not proven. The inference is sound but

not
complete.
The
Lean4
theorem
`analyze_incompleteness`
proves
that
true
invariants
exist
outside
D.
The
Rust
implementation
approximates
the
strongest
invariant
by
structural
analysis,
but
formalizing
that
approximation
and
proving
its
completeness
is
open.

-
- Arithmetic

invariants
outside
D.
The
domain
D
contains
only
predicates
from
base-arrow
guarantees,
conjunction,
and
lifting
through
`first`.
Arithmetic
invariants
are
outside
D
unless
the
signature
encodes
them.
Richer
signatures
enable
richer
invariants,
but
the
domain
boundary

i s
i n h e r e n t
t o
t h e
s t r u c t u r a l
a p p r o a c h .

- Z 3
s u b p r o c e s s .
T h e
R u s t
p r o t o t y p e
i n v o k e s
Z 3
v i a
s u b p r o c e s s
r a t h e r
t h a n
n a t i v e
b i n d i n g s ,
i n c u r r i n g
s e r i a l i z a t i o n
o v e r h e a d .
A
n a t i v e
i n t e g r a t i o n
w o u l d
i m p r o v e
p e r f o r m a n c e
b u t
n o t
s o u n d n e s s .

- F u t u r e
d i r e c t i o n s .
(1)
F o r m a l i z i n g

t h e
l o o p
i n v a r i a n t
d i s c o v e r y
i n
L e a n 4 .
(2)
E x t e n d i n g
t h e
r e f i n e m e n t
l a n g u a g e
b e y o n d
s t r u c t u r a l
p r o p e r t i e s

- •
- •
- •

a r i t h m e t i c
a n d
r e l a t i o n a l
r e f i n e m e n t s .
(3)
E x t e n d i n g
l o o p
i n f e r e n c e
t o
n e s t e d
l o o p s
a n d
m u l t i p l e
f e e d b a c k
c h a n n e l s .
(4)
I n t e g r a t i n g
t h e
R u s t
i n f e r e n c e

engine
with
a
proof
assistant.
(5)
Allowing
shared
sub-graphs
(DAG
structure)
for
proof
reuse.

9.
Conclusion
We
have
presented
an
annotation-free
invariant
inference
mechanism
for
lambda-free
arrow
graphs.
Refinements
flow
from
base-arrow
signatures
through

g r a p h
c o m b i n a t o r s
w i t h o u t
p e r - p r o g r a m
a n n o t a t i o n s ,
a n d
l o o p
i n v a r i a n t
s c h e m a s
a r e
i n f e r r e d
b y
s t r u c t u r a l
a n a l y s i s
o n
t h e
g r a p h
a n d
v e r i f i e d
a g a i n s t
b a s e
a r r o w
c o n t r a c t s

-
-
-

w i t h o u t
u s e r - s u p p l i e d
i n v a r i a n t s .
T h e
c o m p o s i t i o n a l
p r o p a g a t i o n
i s
f o r m a l l y
p r o v e n
s o u n d
i n

Lean4
(inferred_postcondition_sound_full),
strictly
stronger
than
the
trivial
floor
(floor_does_not_imply_inferred),
and
the
existence
of
undetectable
invariants
outside
D
is
proven
(analyze_incompleteness).
The
loop-specific
strengthening
is
implemented
and
empirically
validated
but
not
formally
proven.
The
empirical
evaluation
demonstrates

100%
accuracy
on
72
non-trivial
benchmark
cells:
every
property
the
graph
should
prove
is
proven
(26/26),
and
every
property
it
should
not
is
correctly
reported
as
unknown
(46/46),
with
zero
false
positives.
This
contrasts
with
37

annotations
required
by
Liquid
Types
for
19
properties.
We
have
honestly
characterized
the
boundary:
the
compositional
propagation
is
formally
proven
sound;
the
loop-specific
strengthening
is
implemented
and
empirically
validated
but
not
formally
proven;
and
complex
non-structural

i n v a r i a n t s
a r e
n o t
i n f e r a b l e
b y
t h i s
m e c h a n i s m .
T h e
v i s i o n
i s

-
- p r o o f
- c a r r y i n g

c o m p u t a t i o n
w i t h
m i n i m a l
s y n t a c t i c
o v e r h e a d :
t h e
p r o g r a m m e r
w r i t e s
t h e
g r a p h ,
t h e
l a n g u a g e
d e s i g n e r
s u p p l i e s
t h e
s i g n a t u r e ,
a n d
t h e
c h e c k e r
i n f e r s
t h e
r e f i n e m e n t s
a n d
i n v a r i a n t s

by
structural
reasoning.
The
Curry-Howard
correspondence
is
the
enabling
condition:
the
graph
is
simultaneously
program,
type,
and
proof,
and
the
inference
procedure
reads
the
proof
structure
off
the
graph.
This
is
what
distinguishes
graph-structural
proof
inference

f r o m
a n n o t a t i o n - b a s e d
r e f i n e m e n t
t y p i n g
o r
s e m a n t i c
i n v a r i a n t
s y n t h e s i s :
t h e
p r o o f
i s
a l r e a d y
t h e r e
i n
t h e
w i r i n g

- •
- •
- •

w e
j u s t
h a v e
t o
r e a d
i t .

A p p e n d i x
A
F o u n d a t i o n

- •
- •
- •

A r r o w
G r a p h s
a s
P r o o f

Objects
This
appendix
summarizes
the
proof-theoretic
foundation
established
in
parallel
work
[self-cite],
which
the
inference
mechanism
builds
on.
Full
proofs
are
mechanized
in
Lean4.

A.1 The Curry-Howard Correspondence

We
interpret
arrow
types

a s
p r o p o s i t i o n s
i n
i n t u i t i o n i s t i c
l o g i c :
$ A

r i g h t s q u i g a r r o w
B

m a p s t o
A

v d a s h
B $,
$ A

t i m e s
B

m a p s t o
A

w e d g e
B $,
$ A
•
B

m a p s t o
A

v e e
B $,
$
t o p

m a p s t o

t o p $,

$
bot

mapsto

bot$.
The
typing
rules
and
the
proof
rules
are
notation-for-notation
identical:
Graph
combinator
Type
reading
Proof
reading

|

-
-
-
-
-
-
-
-
-
-
-
-
-
-
-

-
-
-

```
A $
$ A

v d a s h
A $
( i d e n t i t y )
c o m p $ ( G ,
H ) $
$ A

r i g h t s q u i g a r r o w
C $
c u t :
$ A

v d a s h
B ,
B

v d a s h
C

R i g h t a r r o w
A

v d a s h
C $
f i r s t $ ( G ) $
$ A

t i m e s
C

r i g h t s q u i g a r r o w
B

t i m e s
C $
```

$
wedge$-functoriality
left(G)
$A

- C

rightsquigarrow
B

- C$
$
vee$-functoriality
loop(G)
$A

rightsquigarrow
B$
trace
/
fixpoint
invariant
arr$(
varphi)$
$A

rightsquigarrow
B$
axiom
(primitive
inference
step)

-
- Theorem
(Typing
$
Leftrightarrow$
Proof,
Lean4).

- For

every

arrow

graph

G

and

types

$A,

B$:

$
Gamma

vdash

G

A

rightsquigarrow

B$

iff

$
Gamma

vdash

G

Vdash

A

vdash

B$.

The

derivations

are

structurally

identical.

The

maps

`TyOf.toPrOf`

and
PrOf.toTyOf
are
mutually
inverse
bijections
(ProofGraphs.lean).*

A.2
Subject
Reduction

-
- Theorem
(Subject
Reduction,
Lean4).
  - If
$
vdash
G
A

rightsquigarrow
B$
and
$G

cdot
a

Longrightarrow
v$
with
$a
:

A $,
t h e n
$ v
:
B $
(P r o o f G r a p h s . l e a n) . *
T h i s
i s
t h e
k e y
p r o p e r t y
u s e d
b y
t h e
i n f e r e n c e
s o u n d n e s s
p r o o f :
i t
e n s u r e s
t h a t
i n t e r m e d i a t e
v a l u e s
i n
c o m p o s i t i o n s
a r e
w e l l - t y p e d ,
w h i c h
i s
n e e d e d
f o r
t h e
e C o m p
c a s e
o f
i n f e r r e d _ p o s t c o n d i t i o n _ s o u n d _ f u l l .

A.3 Local Cut Elimination

The `CutElim` rewrite relation (`ProofAdvantage.lean`) consists of local rewrites: identity elimination and functor fusion. Each is a single structural step

-
-
-

no substitution, no alpha-conversion, no duplication. Unlike

proof nets, our calculus is

-
- not

linear

-
-
-

first duplicates the side channel; locality comes from the graph structure (no binders), not from linearity.

References

[1] S. Alves, M. Fernandez, I. Mackie.

”A
New
Graphical
Calculus
of
Proofs.”
arXiv:1102.2655,
2011.
[10]
P.
Cousot
and
R.
Cousot.
”Abstract
Interpretation:
A
Unified
Lattice
Model
for
Static
Analysis
of
Programs.”
• POPL
’77*,
pp.
238-252,
1977.
[11]
P.
Cousot
and
R.
Cousot.

”Systematic
Design
of
Program
Analysis
Frameworks.”
• POPL
’79*,
pp.
269-282,
1979.
[16]
C.
Flanagan
and
S.
Qadeer.
”Predicate
Abstraction
for
Software
Verification.”
• POPL
’02*,
pp.
191-202,
2002.
[17]
T.
Freeman
and
F.
Pfenning.
”Refinement
Types
for

ML."
• PLDI
'91*,
pp.
268-277,
1991.
[18]
J.-Y.
Girard.
"Linear
Logic."
• Theor.
Comput.
Sci.*
50:1-102,
1987.
[21]
S.
Graf
and
H.
Saidi.
"Construction
of
Abstract
State
Graphs
with
PVS."
• CAV
'97*,
LNCS
1254,
pp.
72-83,
1997.

[24]
C.
Heunen
and
B.
Jacobs.
"Arrows,
Like
Monads,
Are
Monoids."

- MFPS
- XXII
- ,

ENTCS
158,
pp.
219-236,
2006.
[25]
B.
Jacobs,
C.
Heunen,
I.
Hasuo.
"Categorical
Semantics
for
Arrows."

- J.

Funct.
Program.*
19(3-4):403-438,
2009.
[26]
J.

Hughes.
”Generalising
Monads
to
Arrows.”
• Sci.
Comput.
Program.*
37(1-3):67-111,
2000.
[27]
A.
Joyal,
R.
Street,
D.
Verity.
”Traced
Monoidal
Categories.”
• Math.
Proc.
Camb.
Phil.
Soc.*
119(3):447-468,
1996.
[28]
J.
R.
Hindley
and
J.
P.
Seldin.
• Lambda

- Calculus and Combinators: An Introduction.* 2nd ed., Cambridge University Press, 2008.

[31] S. Lindley, P. Wadler, J. Yallop. "The Arrow Calculus."

- J. Funct. Program.* 20(1):51-69, 2010.

[38] P. M. Rondon, M. Kawaguchi, R. Jhala. "Liquid Types."

• P L D I
’ 0 8 * ,
p p .
1 5 9 - 1 6 9 ,
2 0 0 8 .
[3 9]
J .
L i e l l - C o c k ,
Z .
S h i r a z i ,
S .
S t a t o n .
” T h e
R e l a t i v e
M o n a d i c
M e t a l a n g u a g e . ”
• P r o c .
A C M
P r o g r a m .
L a n g . *
1 0 ,
P O P L ,
A r t i c l e
6 0 ,
2 0 2 6 .
[4 0]
S .
P a d h i ,
R .
S h a r m a ,
T .
M i l l s t e i n .
” L o o p I n v G e n :
A
L o o p
I n v a r i a n t

Generator
based
on
Precondition
Inference."
arXiv:1707.02029,
2017.
[41]
N.
Vazou,
E.
L.
Seidel,
R.
Jhala,
D.
Vytiniotis,
S.
Peyton-Jones.
"Refinement
Types
for
Haskell."
• ICFP
'14*,
pp.
269-282,
2014.
[42]
C.
Kurashige,
R.
Ji,
A.
Giridharan,
M.

Barbone,
D.
Noor,
S.
Itzhaky,
R.
Jhala,
N.
Polikarpova.
"CCLemma:

E

Graph
Guided
Lemma
Discovery
for
Inductive
Equational
Proofs."

- Proc.

ACM
Program.
Lang.*
8,
ICFP,
Article
264,
2024.

[43]
M.
Acclavio,
R.
Horne,
L.
Straßburger.
"An

Analytic
Propositional
Proof
System
on
Graphs.”
- Log.
Methods
Comput.
Sci.*
18(4),
2022.